

Cloud-Based Databases

Bigtable

(Based on the paper: Bigtable: A distributed storage system for structured data, 2006)

What is Bigtable

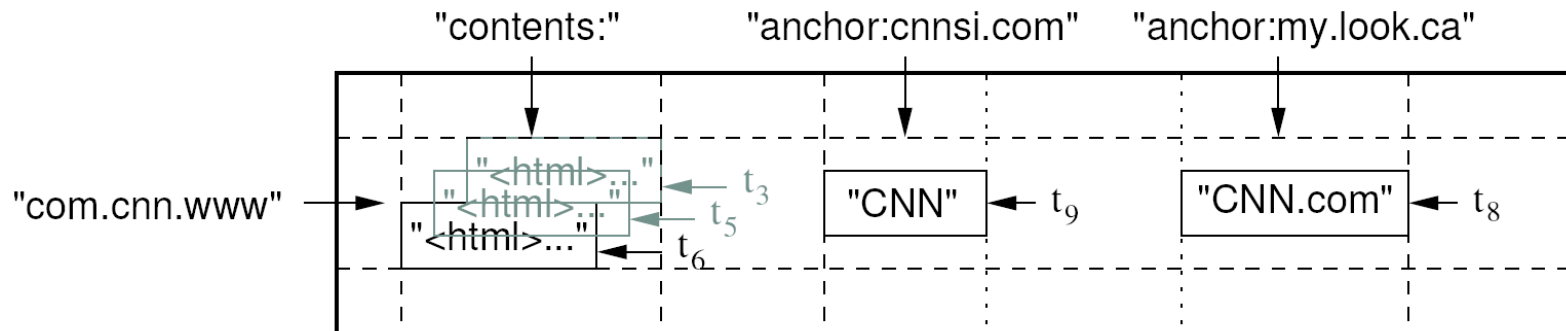
- Bigtable is a compressed, high performance, proprietary data storage system built on Google File System, Chubby Lock Service, SSTable (log-structured storage) and a few other Google technologies.
- Bigtable development began in 2004.
- On May 6, 2015, a public version of Bigtable was made available as a service.
- Bigtable also underlies Google Cloud Datastore, which is available as a part of the Google Cloud Platform.
- Scalability and better control of performance characteristics.

What is Bigtable

- It is now used by a number of Google applications, such as:
 - Web indexing,
 - MapReduce, which is often used for generating and modifying data stored in Bigtable,
 - Google Maps,
 - Google Book Search, "My Search History",
 - Google Earth,
 - Blogger.com,
 - Google Code hosting,
 - YouTube, and
 - Gmail.

Data Model

- A table in Bigtable is a sparse, distributed, persistent multidimensional sorted map.
 - Cloud Bigtable tables are sparse; if a cell does not contain any data, it does not take up any space.
- Map indexed by a row key, column key, and a timestamp
 - (row:string, column:string, time:int64) → uninterpreted byte array
- Supports lookups, inserts, deletes
 - Single row transactions only



Rows and Columns

- Rows maintained in sorted lexicographic order
 - Applications can exploit this property for efficient row scans
 - Row ranges dynamically partitioned into tablets
- Columns grouped into column families
 - Column key = *family:qualifier*
 - Column families provide locality hints
 - Unbounded number of columns

Bigtable example

"follows" column family

	Follows			
Row Key	gWashington	jadams	tjefferson	wmckinley
gWashington		1		
jadams	1		1	
tjefferson	1	1		1
wmckinley			1	

Multiple versions

- The table contains one column family, the *follows* family. This family contains multiple column qualifiers.
- Column qualifiers are used as data. This design choice takes advantage of the sparseness of Bigtable table, and the fact that new column qualifiers can be added on the fly.
- The username is used as the row key. Assuming usernames are evenly spread across the alphabet, data access will be reasonably uniform across the entire table.
 - Using load balancing, Bigtable handles uneven loads.

API Examples: Write/Modify

C++ code that uses a `RowMutation` abstraction to perform a series of updates

```
// Open the table
Table *T = OpenOrDie("/bigtable/web/webtable");

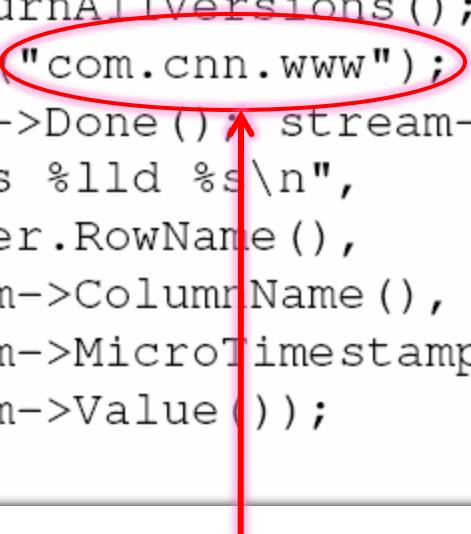
// Write a new anchor and delete an old anchor
RowMutation r1(T, "com.cnn.www");
r1.Set("anchor:www.c-span.org", "CNN");
r1.Delete("anchor:www.abc.com");
Operation op;
Apply(&op, &r1);
```

Apply allows for atomic row modification

API Examples: Read

C++ code that uses a `Scanner` abstraction to iterate over all anchors in a particular row.

```
Scanner scanner(T);
ScanStream *stream;
stream = scanner.FetchColumnFamily("anchor");
stream->SetReturnAllVersions();
scanner.Lookup("com.cnn.www");
for (; !stream->Done(); stream->Next()) {
    printf("%s %s %lld %s\n",
           scanner.RowName(),
           stream->ColumnName(),
           stream->MicroTimestamp(),
           stream->Value());
}
```



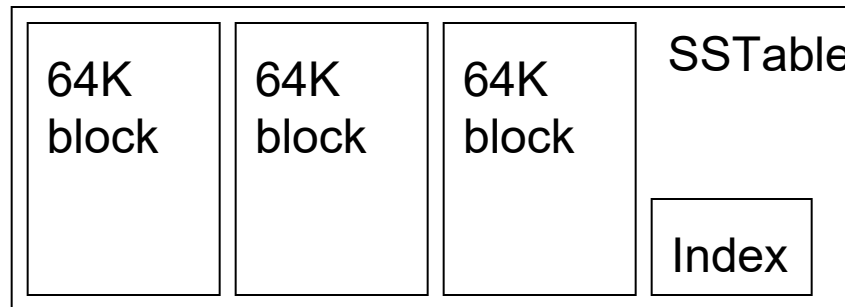
Return sets can be filtered using regular expressions:
anchor: com.cnn.*

Bigtable Building Blocks

- GFS (Apache Hadoop DFS = HDFS)
- Chubby (Apache ZooKeeper)
 - Chubby uses Paxos
 - Zookeeper uses ZAB (ZooKeeper's Atomic Broadcast)
- SSTable
 - **SSTable** (Sorted Strings Table) stores a set of immutable row fragments in sorted order based on row keys.
 - **SSTable** files of a column family are stored in its respective column family directory.
 - The data in a **SSTable** is organized in six types of component files

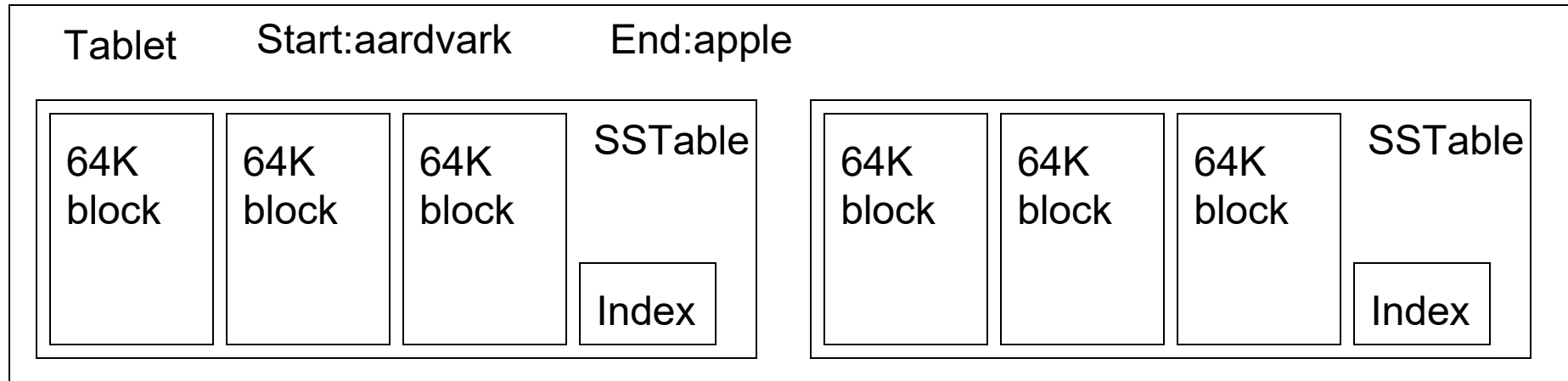
SSTable

- Basic building block of Bigtable
- On-disk file format representing a map from string to string
- Persistent, ordered immutable map from keys to values
 - Stored in GFS
- Sequence of blocks on disk plus an index for block lookup
 - Can be completely mapped into memory
- Supported operations:
 - Look up value associated with key
 - Iterate key/value pairs within a key range



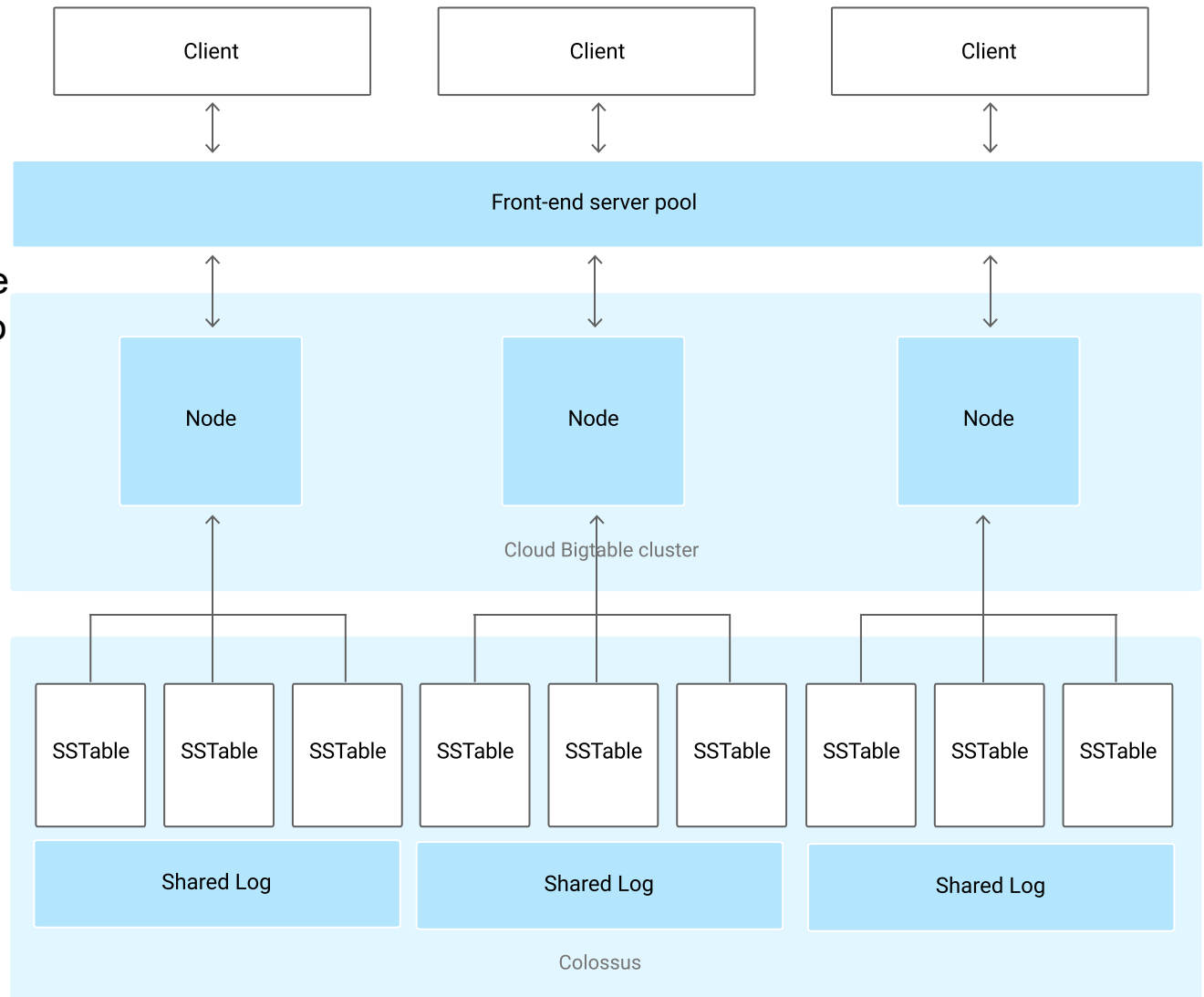
Tablet

- Dynamically partitioned range of rows
- Built from multiple SSTables



Architecture

- Client (with library API)
- Single master server
- Tablet servers (node)
- Tablet servers are organized into a Bigtable cluster, which belongs to a Bigtable instance.
- Table is sharded into blocks of contiguous rows, called tablets.
- Tablets are stored on Colossus, Google's file system, in SSTable format.
- Writes are stored in the file system's (Colossus) shared log.



Bigtable Master

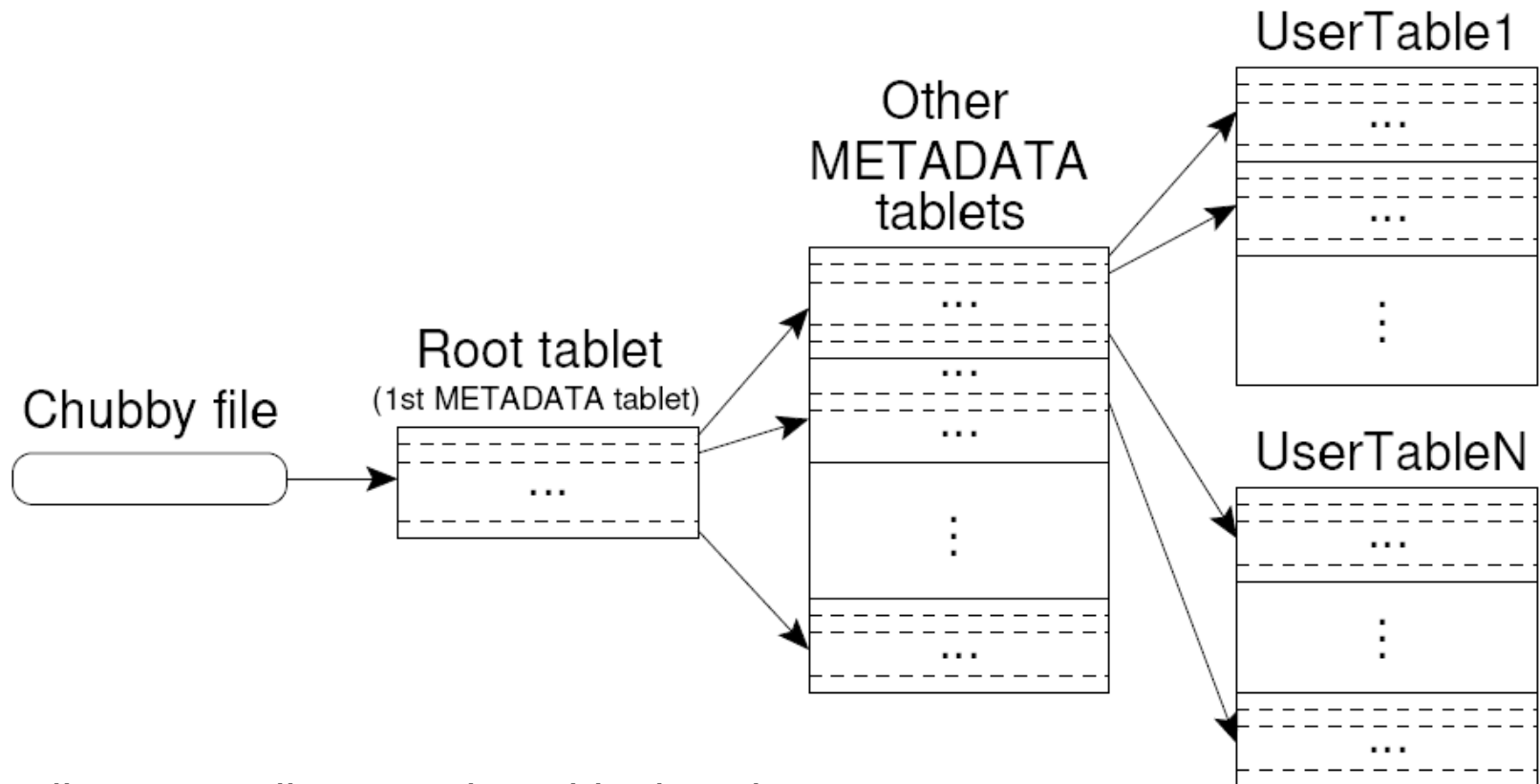
- Assigns tablets to tablet servers
- Detects addition and expiration of tablet servers
- Balances tablet server load. Tablets are distributed randomly on nodes of the cluster for load balancing.
- Handles garbage collection
- Handles schema changes

Bigtable Tablet Servers

- Each tablet server manages a set of tablets
 - Typically between ten to a thousand tablets
 - Each 100-200 MB by default
- Handles read and write requests to the tablets
- Splits tablets that have grown too large

Tablet Location

- Using a 3-level B+-tree
- The first level is a file stored in Chubby that contains the location of the root tablet.
- The root tablet contains the location of all tablets in a special METADATA table.
- Each METADATA tablet contains the location of a set of user tablets.
- The root tablet is just the first tablet in the METADATA table, but is treated specially—it is never split—to ensure that the tablet location hierarchy has no more than three levels.



Upon discovery, clients cache tablet locations

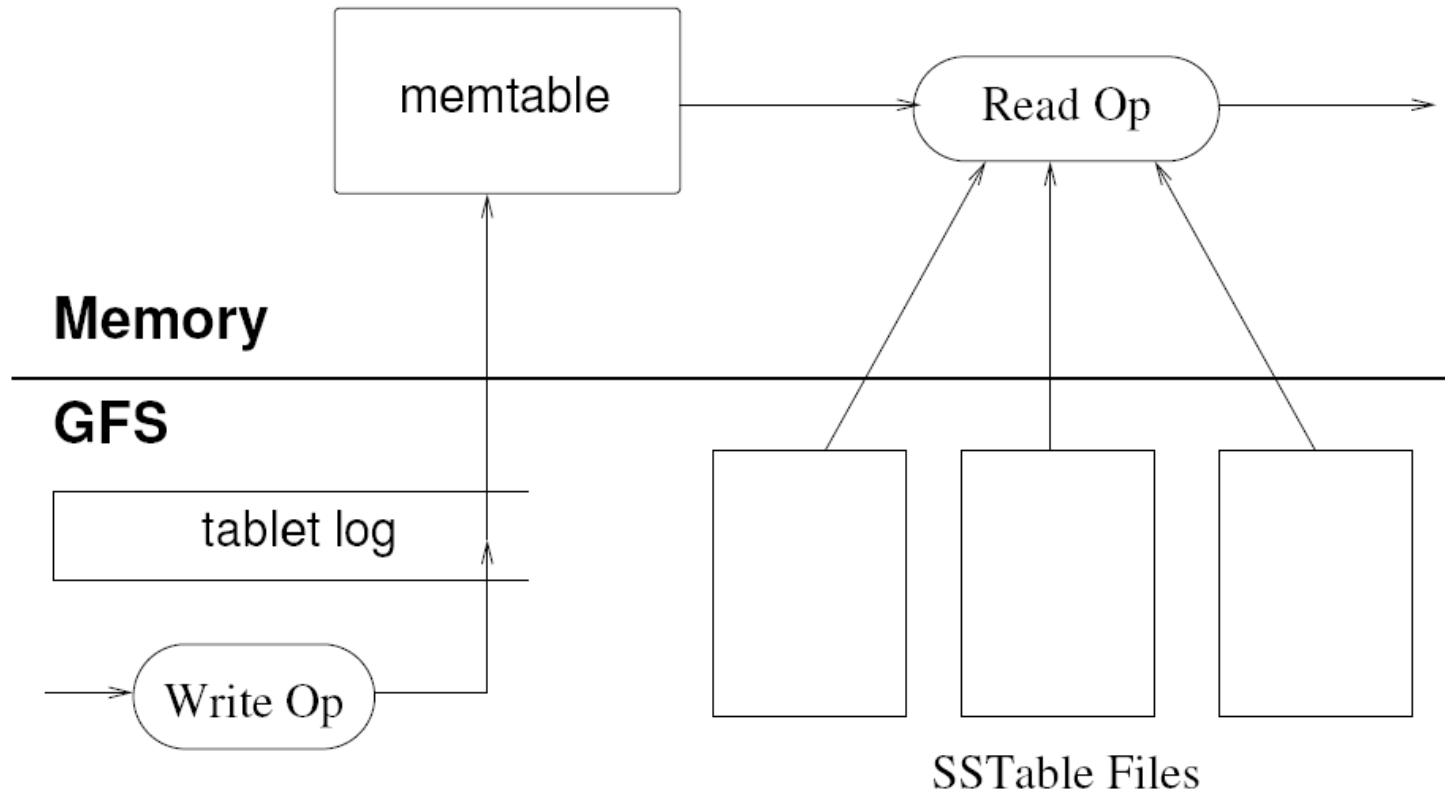
Tablet Assignment

- Master keeps track of:
 - Set of live tablet servers
 - Assignment of tablets to tablet servers
 - Unassigned tablets
- Each tablet is assigned to one tablet server at a time
 - Tablet server maintains an exclusive lock on a file in Chubby
 - Master monitors tablet servers and handles assignment
- Changes to tablet structure
 - Table creation/deletion (master initiated)
 - Tablet merging (master initiated)
 - Tablet splitting (tablet server initiated)

Tablet Serving

Updates are committed to a commit log that stores redo records.

The recently committed updates are stored in memory in a sorted buffer called a memtable; the older updates are stored in a sequence of SSTables.



“Log Structured Merge Trees”

Tablet Representation

- When a write operation arrives at a tablet server, the server checks that it is well-formed, and that the sender is authorized to perform the mutation.
- Authorization is performed by reading the list of permitted writers from a Chubby (which is almost always a hit in the Chubby client cache).
- A valid mutation is written to the commit log.
- Group commit is used to improve the throughput of lots of small mutations.
- After the write has been committed, its contents are inserted into the memtable.

Tablet Representation

- When a read operation arrives at a tablet server, it is similarly checked for well-formedness and proper authorization.
- A valid read operation is executed on a merged view of the sequence of SSTables and the memtable.
- Since the SSTables and the memtable are lexicographically sorted data structures, the merged view can be formed efficiently.

Compactions

- Minor compaction
 - Converts the memtable into an SSTable
 - Reduces memory usage and log traffic on restart
- Merging compaction
 - Reads the contents of a few SSTables and the memtable, and writes out a new SSTable
 - Reduces number of SSTables
- Major compaction
 - Merging compaction that results in only one SSTable
 - No deletion records, only live data

Lock server

- Chubby
 - Highly-available & persistent distributed lock service
 - Five active replicas; one acts as master to serve requests
- Chubby is used to:
 - Ensure there is only one active master
 - Store bootstrap location of BigTable data
 - Discover tablet servers
 - Store BigTable schema information
 - Store access control lists
- If Chubby dies for a long period of time... Bigtable dies too....
 - But this almost never happens...

Optimizations

- Log of tablets in the same server are merged in one log per tablet server (node)
- Locality groups: separate SSTables are created for each locality group of column families that form the locality groups.
- Use efficient and lightweight compression to reduce the size of SSTable blocks. Since data are organized by column(s) very good compression is achieved (similar values together)
- Tablet servers use two levels of caching
- Bloom filters are used to skip some SSTables and reduce read overhead.

More NoSQL databases

- **Wide Column Store / Column Families**

HBase, Cassandra, Hypertable, Cloudata, Cloudera, **Amazon SimpleDB**

- **Document Stores**

CouchDB, MongoDB, Terrastore, ThruDB, OrientDB, RavenDB, Citrusleaf, SisoDB

- **Key Value / Tuple Store**

Azure Table Storage, MEMBASE, Riak, Redis, Chordless, GenieDB, Scalaris, Tokyo Cabinet / Tyrant, Keyspace

Berkeley DB, **MemcacheDB**, Faircom C-Tree, Mnesia, LightCloud, Hibari, HamsterDB, STSdb, Pincaster, RaptorDB

- **Eventually Consistent Key Value Stores**

Amazon **Dynamo**, **Voldemort**, Dynomite, KAI

- **Graph Databases**

Neo4J, Infinite Graph, Sones, InfoGrid, HyperGraphDB, Trinity, AllegroGraph, Bigdata, DEX, OpenLink, Virtuoso, VertexDB, FlockDB

- **Object Databases**

db4o, Versant, Objectivity, Gemstone, Progress, Starcounter, Perst, Caching, ZODB, NEO, PicoLisp, Sterling

- More and more databases

References

- **Overview of Bigtable:**

<https://cloud.google.com/bigtable/docs/overview>

- Chang, F., Dean, J., Ghemawat, S., Hsieh, W. C., Wallach, D. A., Burrows, M., ... & Gruber, R. E. (2008). **Bigtable: A distributed storage system for structured data.** *ACM Transactions on Computer Systems (TOCS)*, 26(2), 1-26.

- **DB-Engines Ranking of Key-value Stores**

<https://db-engines.com/en/ranking/key-value+store>

End of Slides